Final Project Presentation on

# Solving TAPF via Poisson Potential Fields and Heuristic Word Reduction in Permutation Groups

Boxiang (William) Fu, F.D.C. Plato

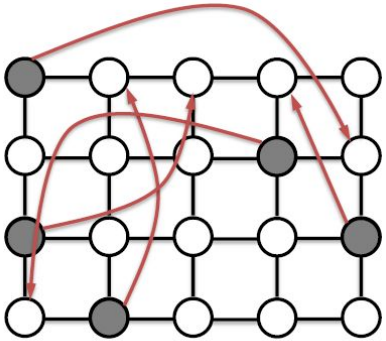
16-891 (Spring 2026) Multi-Robot Planning and Coordination
Carnegie Mellon University

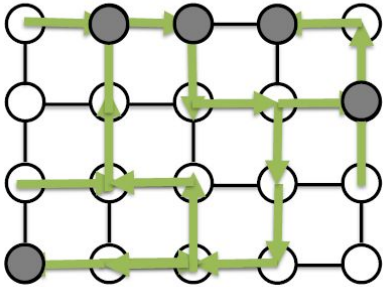
**04/20/2026**

# Overview

- **MAPF**: Multi-agent path finding from pre-assigned starts to goals
- **TAPF**: Assigning the starts and goals + MAPF
- **Problem**: MAPF/TAPF as graph traversal problem
- **Alternative**: MAPF/TAPF as **constrained word problem**
- **How**: Recasting problem as **Group Actions** of **Automorphism Groups**

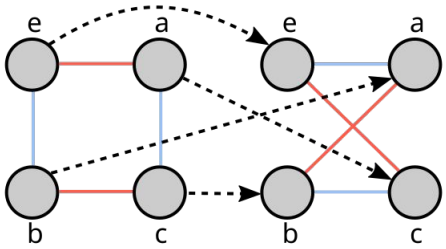
  **Generated by Graph Edge Transpositions**





# Group Theory

- **Preliminaries**: Graph $G = (V, E)$, Agents A = {1, … , n}, Starts $S$, Goals $G$
- **Group** ($H$): Set with a binary operation that satisfies associativity, identity & inverse
- **Group Action** ($H \curvearrowright V$): A function that maps a group $H$ and a set $V$ to $V$ itself
- **Automorphism Group** (Aut($V$)): All bijective functions (permutations) from $V$ to $V$
- **Generated by Graph Edge Transpositions** ($H = \langle (u\ v) \mid (u, v) \in E \rangle$): Composition of graph edges (adjacent transpositions) produce all members of group $H$
- **Constrained Word Problem**: Finding a constrained sequence of generator elements in $H$ that compose together to generate the $S$ to $G$ permutation




$\phi(e) = e$
$\phi(a) = c$
$\phi(b) = a$
$\phi(c) = b$
$\phi = (acb)$

# Group Theory (Formally)

A *group* $(G, *)$ is a set $G$ together with an operation

$$*: G \times G \to G$$

such that

(a) $g * (h * k) = (g * h) * k$ for all $g, h, k \in G$;

(b) there exists an element $e \in G$ such that

$$g * e = e * g = g \qquad \text{for all} \quad g \in G;$$

($e$ is said to be an *identity element of* $G$);

(c) for every $g \in G$ there exists an element $g^{-1} \in G$ such that

$$g * g^{-1} = g^{-1} * g = e;$$

($g^{-1}$ is called an *inverse of* $g$).

Let $G$ be a group, $X$ a set. A $G$-*action on* $X$ is a function

$$\begin{aligned} G \times X &\longrightarrow X \\ (g, x) &\longmapsto g \cdot x \end{aligned}$$

such that

(a) $e \cdot x = x$ for all $x \in X$;

(b) $(gh) \cdot x = g \cdot (h \cdot x)$ for all $g, h \in G$ and all $x \in X$.

In this situation, we say that $G$ *acts on* $X$ and write $G \circlearrowright X$.

# Related Works

- MAPF algorithms such as Joint-State A*, CBS, PBS, PIBT, LaCAM, etc
- TAPF algorithms such as CBM, CBS-TA, ITA-CBS, etc
- Group partitioning into Young subgroups via min-cuts (James, 1984)
- Symmetry groups and automorphism groups (Ghitza, 2021)
- Group actions (Rotman, 1965)
- Word problem solving via pebble motion (Kornhauser et al., 2015)
- Planning via navigation functions (Rimon & Koditschek, 1988)
- Poisson potential fields (Poisson, 1823)
- Dynamic path planning via perturbed Laplace's equation (My 16-832 Project)

# Methodology (Local Heuristic)

- **Idea**: Greedy (fast) solver for constrained pebble moving problem
- Pick a single agent (pebble) to move
- Rank agent moves based on local heuristic (potential field)
- Move agent in preferred cell in the downward direction of potential field
- If the preferred cell is occupied, recursively displace the blocking chain to free the move
- If displacement is impossible, move to less preferred cells
- Restrict moves to non-backtracking moves to prevent oscillations

# Pseudocode (Local Heuristic)

**Algorithm 1** Low-Level Pebble Motion Step

```
L: current agent locations,  T: transposition sequence
Get legal moves 𝒩 of agent i
Rank 𝒩 and let best move be v
if v is free then
    Move i to v, update L, append (i, v) to T
else
    Let j be the agent blocking v
    if blocking agents can be recursively moved to free v then
        Recursively move blocking agents, update L, append transpositions to T
        Move i to v, update L, append (i, v) to T
    else if i has a valid non-backtracking move u then
        Move i to u, update L, append (i, u) to T
    else
        return failure
    end if
end if
```

# Methodology (Global Guidance)

- **Idea**: No initial agent-goal allocation – let physics do the allocation for us
- Model agent locations as positively charged source particles
- Model goal locations as negatively charged sink particles
- Model map boundaries as Neumann boundary conditions (i.e. normal derivative is 0)
- Solve for Poisson's equation in free-space ($\nabla^2\varphi(x) = \rho$)
- Field lines of Poisson potential field will flow towards goals
- High-level search tree for agent-goal allocation
- Re-solve Poisson's equation whenever a goal is reached
- When agent-goal allocation is unsolvable, backtrack via UCB in search tree

# Pseudocode (Global Guidance)

**Algorithm 2** High-Level TAPF Solver

```
Initialize locations L ← S, moves T ← ∅, active agents A ← {1, ..., n}
Precompute potential field φ from active starts to active goals
while L ≠ G and time not exceeded do
    Choose agent i from A
    if L[i] ∈ G then
        Mark i inactive, update active goals, recompute φ
        Commit current moves T to the search tree
        continue
    end if
    if PebbleMotion(i, L, T, φ) succeeds then
        Update L and T
    else
        if backtrack by UCB succeeds then
            Restore previous state
            continue
        else
            break
        end if
    end if
end while
return Convert T to agent paths
```

# Methodology & Pseudocode (Path Conversion)

- **Idea**: Convert pebble motion (group generator elements) into agent paths

**Algorithm 3** Convert Word Sequence to Agent Paths

**Require:** Transposition deque $T$, number of agents $n$, start locations $S$
**Ensure:** Paths $\Pi = \{\pi_1, \ldots, \pi_n\}$

```
Initialize timestep deque 𝒯 ← ∅, current timestep τ ← ∅, used-agent set A ← ∅, used-
destination set D ← ∅
while T ≠ ∅ do
    Pop (i, (v_end, v_start)) from T
    if i ∈ A or v_end ∈ D then
        Push τ into 𝒯 and reset τ, A, D
    end if
    Add (i, v_end, v_start) to τ
    A ← A ∪ {i},    D ← D ∪ {v_end}
end while
Initialize each path π_i ← [s_i]
for each timestep τ in chronological order do
    Let M be the set of agents moving in τ
    for each (i, v_end, v_start) ∈ τ do
        Append v_end to π_i
    end for
    for each agent i ∉ M do
        Append the previous location of π_i to π_i
    end for
end for
return Π
```

# Methodology & Pseudocode (Poisson Solver)

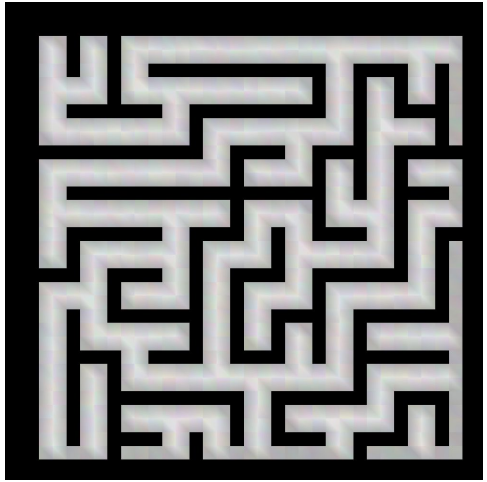
- **Idea**: Fast Poisson's equation solver in free-space using a sparse linear solver

**Algorithm 4** Poisson Solver

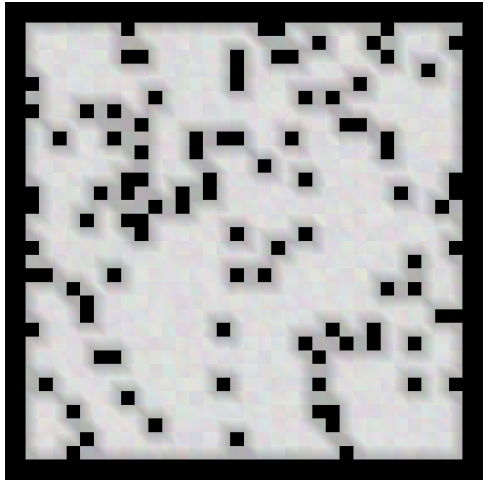
```
Pad map with obstacle boundary
Remove anchor-free connected free-space components
Flatten grid to indices k = r · w + c
for each free 4-neighbor pair (p, q) do
    Set A_pq ← −1
end for
for each cell p do
    if p is free and not fixed then
        Set A_pp ← deg(p)
    else
        Set A_pp ← 1
    end if
end for
Set b_p ← +α for starts, b_p ← −α for goals
Solve Aφ = b
Reshape, zero obstacle cells, and crop boundary
return φ
```

# Baselines

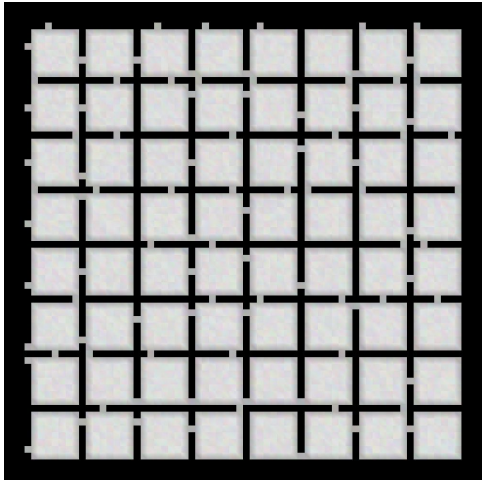
- Three maps from Moving AI Lab
- Agents and goals generated randomly in free-space
- Number of agents increased from 10 to 200 in 10-agent increments



maze-32-32-2



random-32-32-10

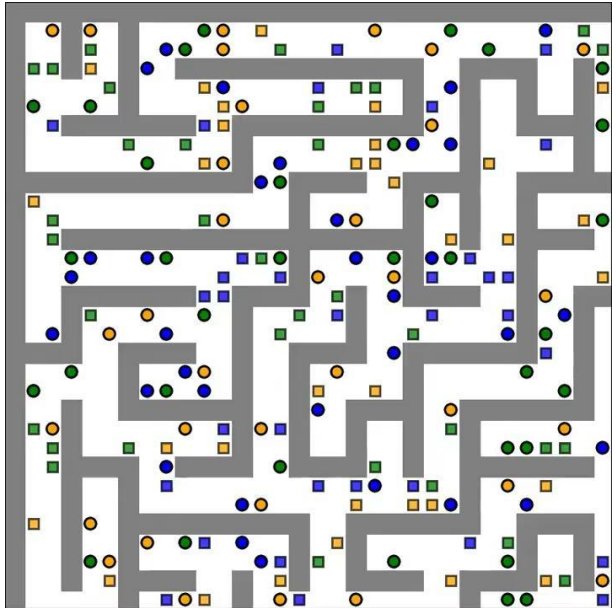


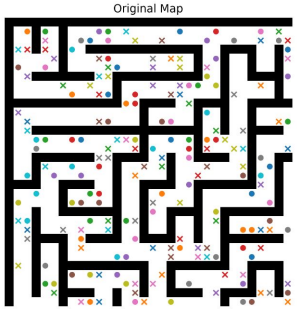
room-64-64-8

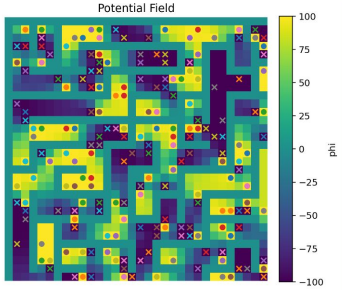
# Results (maze-32-32-2)

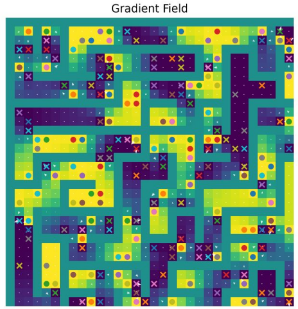
Table 1: Results on `maze-32-32-2`

| # Agents | Cost | CPU Time (s) | Node Count |
|---|---|---|---|
| 10 | 1190 | 0.018 | 13 |
| 20 | 3360 | 0.056 | 37 |
| 30 | 5310 | 0.089 | 59 |
| 40 | 5760 | 0.163 | 105 |
| 50 | 7700 | 0.196 | 107 |
| 60 | 10260 | 0.173 | 110 |
| 70 | 11130 | 0.219 | 142 |
| 80 | 10800 | 0.326 | 204 |
| 90 | 15030 | 0.351 | 219 |
| 100 | 19600 | 0.439 | 248 |
| 110 | 17050 | 0.392 | 247 |
| 120 | 28080 | 0.578 | 364 |
| 130 | 26520 | 0.603 | 359 |
| 140 | 38920 | 0.846 | 510 |
| 150 | 41250 | 0.893 | 526 |
| 160 | 33120 | 0.719 | 441 |
| 170 | 70550 | 1.426 | 898 |
| 180 | 63720 | 1.315 | 822 |
| 190 | 43700 | 0.961 | 555 |
| 200 | 37000 | 0.800 | 465 |





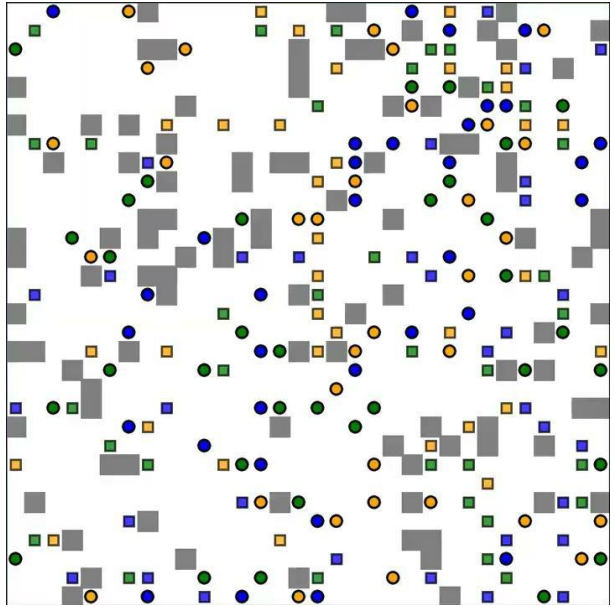



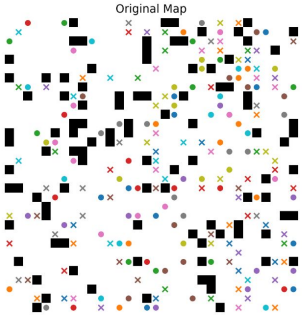



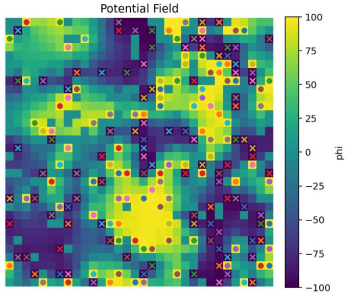
# Results (random-32-32-10)

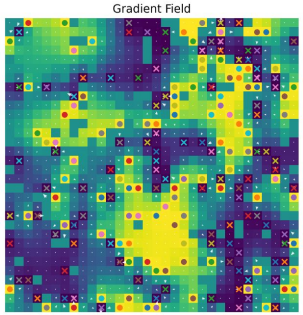
Table 2: Results on `random-32-32-10`

| # Agents | Cost | CPU Time (s) | Node Count |
|---|---|---|---|
| 10 | 420 | 0.024 | 11 |
| 20 | 1140 | 0.059 | 26 |
| 30 | 1560 | 0.086 | 37 |
| 40 | 3120 | 0.136 | 57 |
| 50 | 1700 | 0.154 | 64 |
| 60 | 3060 | 0.181 | 74 |
| 70 | 2800 | 0.213 | 87 |
| 80 | 4240 | 0.271 | 111 |
| 90 | 7560 | 0.387 | 145 |
| 100 | 8500 | 0.402 | 145 |
| 110 | 7260 | 0.392 | 152 |
| 120 | 9600 | 0.518 | 206 |
| 130 | 8580 | 0.545 | 217 |
| 140 | 11900 | 0.596 | 236 |
| 150 | 12300 | 0.666 | 262 |
| 160 | 12000 | 0.651 | 258 |
| 170 | 12750 | 0.722 | 282 |
| 180 | 18000 | 0.834 | 317 |
| 190 | 14250 | 0.801 | 305 |
| 200 | 16800 | 0.879 | 337 |





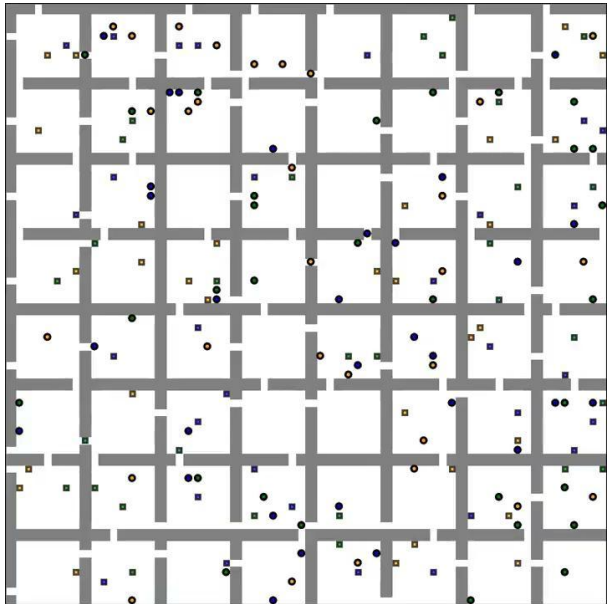



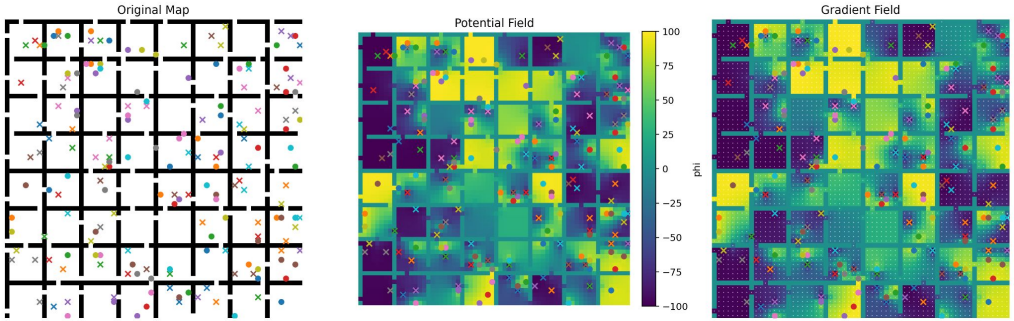



# Results (room-64-64-8)

Table 3: Results on room-64-64-8

| # Agents | Cost | CPU Time (s) | Node Count |
|---|---|---|---|
| 10 | 1140 | 0.064 | 11 |
| 20 | 4280 | 0.241 | 32 |
| 30 | 5130 | 0.332 | 43 |
| 40 | 10920 | 0.923 | 116 |
| 50 | 8900 | 0.532 | 69 |
| 60 | 15360 | 1.541 | 200 |
| 70 | 19530 | 0.932 | 116 |
| 80 | 22320 | 0.975 | 121 |
| 90 | 19260 | 0.980 | 122 |
| 100 | -1 | 30.003 | 4483 |
| 110 | 41250 | 1.559 | 186 |
| 120 | 25080 | 1.445 | 173 |
| 130 | 36140 | 1.976 | 234 |
| 140 | 57260 | 2.732 | 330 |
| 150 | 35250 | 1.989 | 236 |
| 160 | 44480 | 2.347 | 281 |
| 170 | -1 | 30.002 | 4100 |
| 180 | 32220 | 2.133 | 256 |
| 190 | 42560 | 2.294 | 267 |
| 200 | 71800 | 3.237 | 391 |





# Evaluation

- **Complete**: No
- **Optimal**: No
- **Speed**: Very fast
- Executor can automatically account for agent delays
  - If an agent is delayed, simply rerun path converter
  - Regroups pebble motion permutations into new blocks of agent paths
- Multiple agents can sometimes be attracted to the same sink. When the first agent occupies it, the recalculated potential field can cause the agents to travel great distances

# Future Work

- Explore the method's theoretical properties
  - Intuition: Variations of this method could be probabilistic complete
  - Why: UCB incurs sub-linear regret – this class of algorithms tend to be probabilistic complete
  - Why: Physics-motivated – Maxwell's equations dictate that field lines eventually connect sources to sinks
- Multi-team TAPF with different agent and goal types
  - Intuition: Different potential fields for different group of agents
- Further group-theoretic MAPF/TAPF solvers

# Conclusion

- Recast MAPF/TAPF problems as group action constrained word problem
- Solver combine global Poisson-field guidance with local greedy pebble motion
- High-level search tree with UCB backtracking for agent-goal allocation
- Fast runtime across multiple Moving AI benchmark maps
- No completeness or optimality guarantees
- Physics-inspired guidance and group-action structure can be a promising alternative to standard graph-search methods
- Future work on strengthening theory, improving allocation stability, and extend to multi-team TAPF

# Thank You!

https://github.com/williamfbx/Group-Action-MAPF